

Text Retrieval and Search Engines



Lecture 9 – The Language Modeling Approach

Search

Some slides adapted from the companion website to the
"Introduction to IR" book by Manning, Raghavan and Schütz & Victor Lavrenko

Statistical Language Models

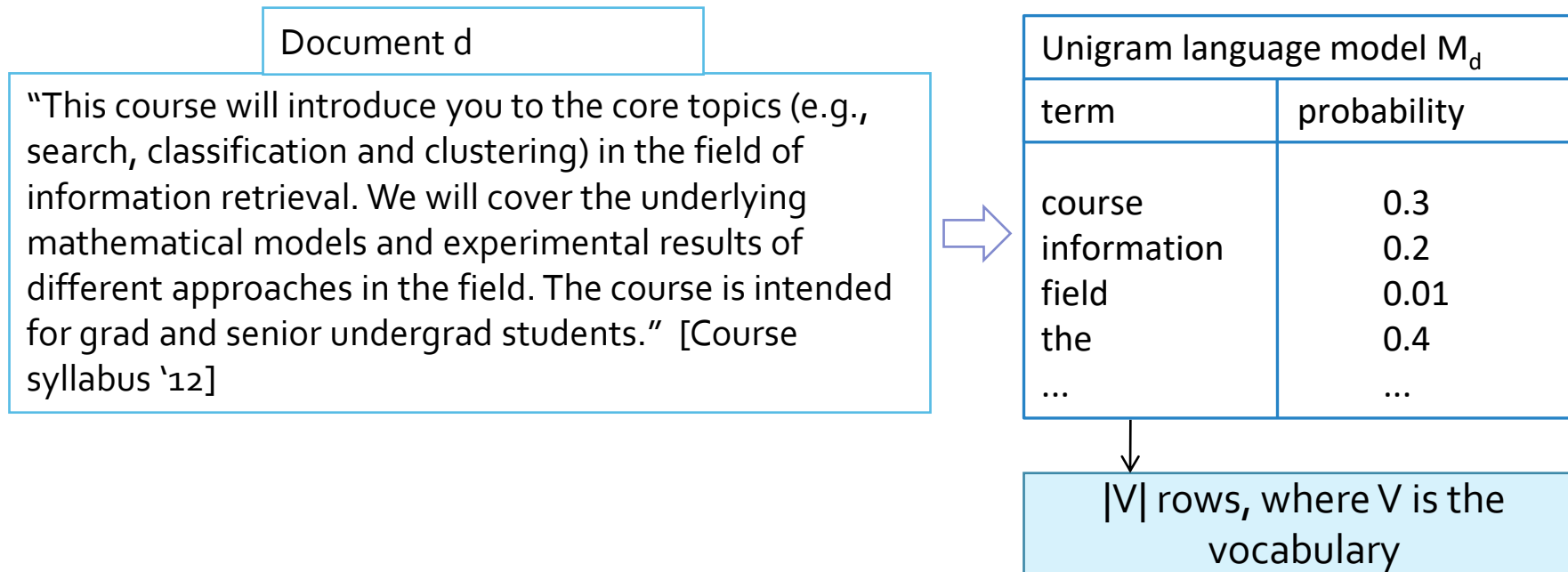
- Probability distribution of sequences of terms
 - M is a model
 - $P(\text{"Today is Thursday"} \mid M) = 0.001$
 - $P(\text{"Today Thursday is"} \mid M) = 0.0000001$
 - $P(\text{"Information retrieval is fun"} \mid M) = 0.01$
- Context-dependent

Statistical Language Models

- Probability distribution of sequences of terms
 - M is a model
 - $P(\text{"Today is Thursday"} \mid M) = 0.001$
 - $P(\text{"Today Thursday is"} \mid M) = 0.0000001$
 - $P(\text{"Information retrieval is fun"} \mid M) = 0.01$
- Context-dependent
- Can also be regarded as probabilistic mechanism for “generating” text, thus also called a “generative” model

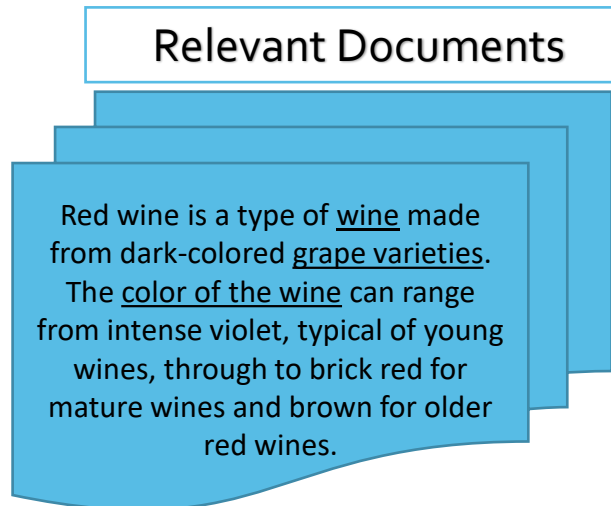
Statistical Language Models

- Probability distribution of sequences of terms



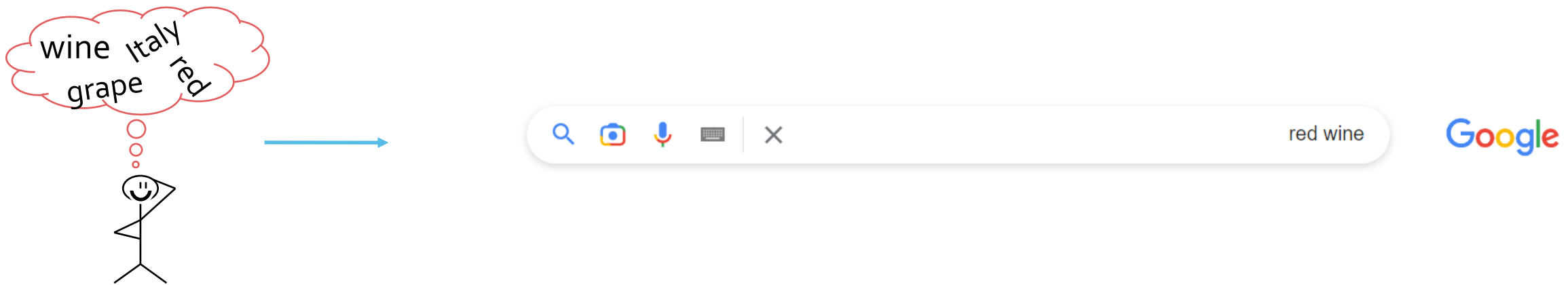
Language Models for Retrieval

- Use LMs to model the process of query generation
 - User thinks of some relevant documents
 - User picks some keywords to use as the query
 - Documents are ranked by the probability that a query would be observed as a random sample from the respective document model



Language Models for Retrieval

- Use LMs to model the process of query generation
 - User thinks of some relevant documents
 - User picks some keywords to use as the query
 - Documents are ranked by the probability that a query would be observed as a random sample from the respective document model

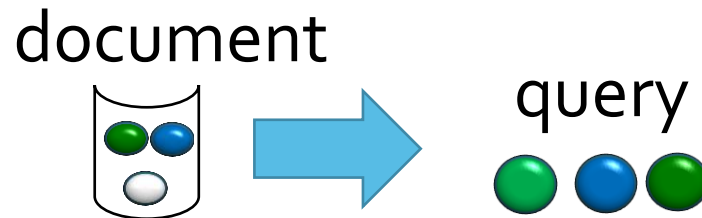


Language Models for Retrieval

1. Infer a language model M_d for each document d_i
 - Each document d in a collection defines a “language”

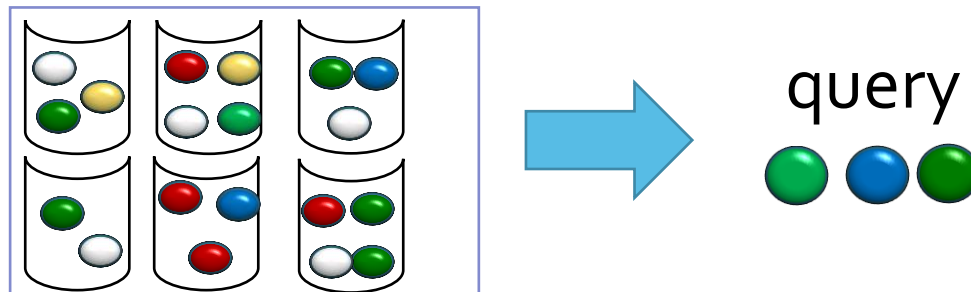
Language Models for Retrieval

1. Infer a language model M_d for each document d_i
 - Each document d in a collection defines a “language”
2. For a query q and each document d_i calculate $P(q \mid M_{d_i})$
 - The probability to generate a query q from M_{d_i}
 - The probability that the author of the document d_i would write q while creating d_i



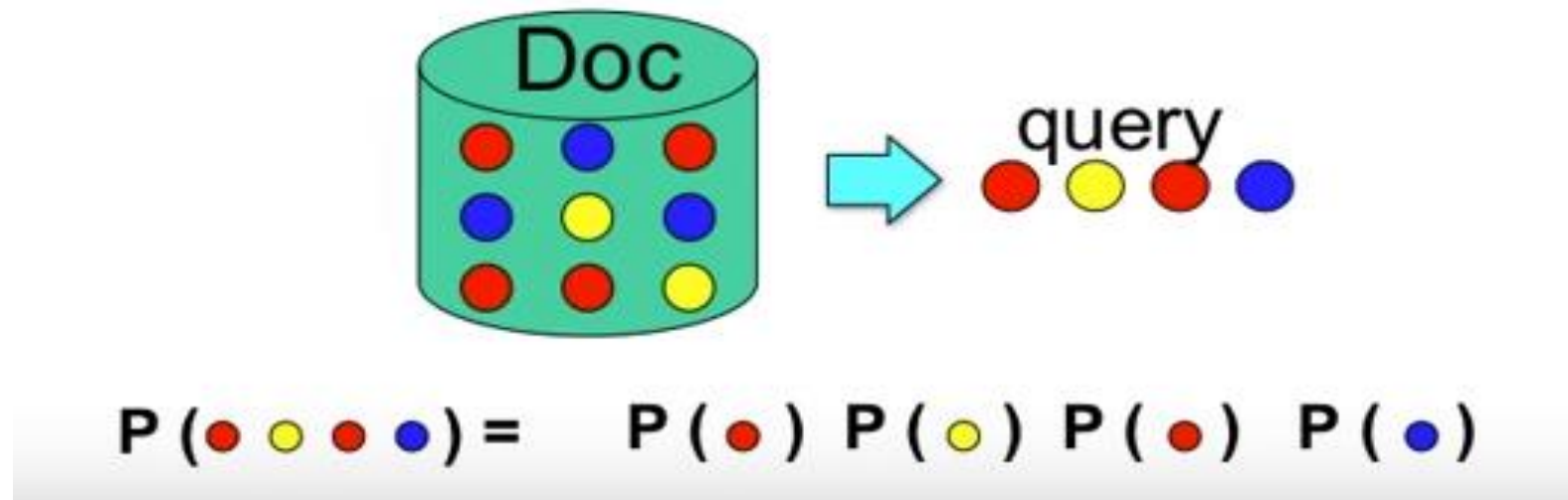
Language Models for Retrieval

1. Infer a language model M_d for each document d_i
 - Each document d in a collection defines a “language”
2. For a query q and each document d_i calculate $P(q \mid M_{d_i})$
 - The probability to generate a query q from M_{d_i}
 - The probability that the author of the document d_i would write q while creating d_i
3. Retrieval: rank documents by $P(q \mid M_{d_i})$



Unigram Language Models

- Words are “sampled” independently of each other
 - Generate text by generating each word **independently**
 - “bag of words” representation
 - Joint probability decomposes into a product of marginals
 - Parameters: $P(t_1) + \dots + P(t_{|V|}) = 1$
 - $|V|$ is the vocabulary size



Query Generation Probability

- “Bag of words” representation – a sample from a multinomial distribution
- N independent trails (N = length of d)
- $|V|$ possible outcomes for each trail (V = vocabulary)
- Each trail outcome has constant probability $P(t_i|M_d)$, $\sum_{t_i \in V} P(t_i|M_d) = 1$
- $$P(d|M_d) = \frac{N!}{tf(t_1 \in d)!tf(t_2 \in d)! \dots tf(t_V \in d)!} P(t_1|M_d)^{tf(t_1 \in d)} P(t_2|M_d)^{tf(t_2 \in d)} \dots P(t_V|M_d)^{tf(t_V \in d)}$$
 - $tf(t_i \in d)$ is the number of occurrences of term t_i in d (raw term frequency)

Query Generation Probability

- “Bag of words” representation – a sample from a multinomial distribution
- N independent trails (N = length of d)
- $|V|$ possible outcomes for each trail (V = vocabulary)
- Each trail outcome has constant probability $P(t_i|M_d)$, $\sum_{t_i \in V} P(t_i|M_d) = 1$
- $$P(d|M_d) = \frac{N!}{tf(t_1 \in d)!tf(t_2 \in d)! \dots tf(t_V \in d)!} P(t_1|M_d)^{tf(t_1 \in d)} P(t_2|M_d)^{tf(t_2 \in d)} \dots P(t_V|M_d)^{tf(t_V \in d)}$$
 - $tf(t_i \in d)$ is the number of occurrences of term t_i in d (raw term frequency)
- How to estimate $P(t_i|M_d)$?
 - Maximum likelihood estimate (MLE): $P_{MLE}(t_i|M_d) = \frac{tf(t_i \in d)}{N}$

Query Likelihood

- Ranking formula

$$p(d|q) = \frac{p(d)p(q|d)}{p(q)} = p(d)p(q|d) \approx p(d)p(q|M_d)$$

- The probability of producing the query given the language model of document d using MLE is:

$$\mathbf{p}_{MLE}(\mathbf{q} | \mathbf{M}_d) \stackrel{\text{def}}{=} \prod_{t \in \mathbf{q}} \mathbf{p}_{MLE}(\mathbf{t} | \mathbf{M}_d) = \prod_{t \in \mathbf{q}} \frac{tf(t \in d)}{|d|}$$

Insufficient Data

- Zero probability $p_{MLE}(t|M_d) = 0$
 - a.k.a sparse data problem
- Happens very frequently with language
- It is incorrect to infer zero probabilities to a document that is missing one or more query terms
 - Especially when dealing with incomplete samples

Sparse Data Problem

- General approach – use background probabilities (the entire collection)
 - A non-occurring term is possible, but no more likely than would be expected by chance in the collection
- If $tf(t \in d) = 0$ then define $p_{MLE}(t|M_d) = p_{MLE}(t|M_C) = \frac{tf(t \in C)}{\sum_{w'} tf(w' \in C)}$
 - C is corpus of documents
 - A “large document”

Jelinek Mercer Smoothing

$$p_{JM}(t | M_d) = (1 - \lambda)p_{MLE}(t | M_d) + \lambda p_{MLE}(t | M_C)$$

- Mixes the probability from the document with the general collection frequency of the word
- Correctly setting λ is very important
 - λ is constant, independent of document, query
- A low value of lambda makes the search “conjunctive-like” – suitable for short queries
- A high value is more suitable for long queries
- Tune to optimize performance

Bayesian Smoothing with Dirichlet Priors

$$p_{Dir}(t | M_d) = \frac{tf(t \in d) + \mu p_{MLE}(t | M_C)}{|d| + \mu}$$

- Equivalent to writing:
 - $p_{Dir}(t | M_d) = (1 - \lambda)p_{MLE}(t | M_d) + \lambda p_{MLE}(t | M_C)$
 - Where $\lambda = \frac{\mu}{|d| + \mu}$
 - μ is Dirichlet prior

Example

- Rank the following documents with respect to the query = “onion soup onion”
- Use JM smoothing with $\lambda = 0.2$

D1: onion vegetable soup vegetable mushroom corn

D2: corn onion soup onion mushroom corn

D3: potato pumpkin tofu potato tofu potato

- First, MLE unigram from the collection - M_C
 - Total of 18 terms, 8 are unique

term	onion	vegetable	soup	mushroom	corn	potato	pumpkin	tofu
tf	3	2	2	2	3	3	1	2
P_{MLE}	3/18	2/18	2/18	2/18	3/18	3/18	1/18	2/18

Example

- MLE unigram from documents

term/ doc.	onion	vegetable	soup	mushroom	corn	potato	pumpkin	tofu
D1	1/6	2/6	1/6	1/6	1/6	0	0	0
D2	2/6	0	1/6	1/6	2/6	0	0	0
D3	0	0	0	0	0	3/6	1/6	2/6

- Smoothed LM ($\lambda = 0.2$)

- $p(\text{onion}|D1) = 0.8 * \left(\frac{1}{6}\right) + 0.2 * \left(\frac{3}{18}\right) = \frac{1}{6}$
- $p(\text{soup}|D1) = 0.8 * \left(\frac{1}{6}\right) + 0.2 * \left(\frac{2}{18}\right) = 0.156$

term/ doc.	onion	vegetable	soup	mushroom	corn	potato	pumpkin	tofu
D1	1/6	0.29	0.156	0.156	1/6	0.033	0.011	0.022
D2	0.3	0.022	0.156	0.156	0.3	0.033	0.011	0.022
D3	0.033	0.022	0.022	0.022	0.033	0.433	0.144	0.29

Example

- Query likelihood
 - $p(q|D1) = p(\text{onion}|D1)p(\text{soup}|D1)p(\text{onion}|D1) = \frac{1}{6} * 0.156 * \frac{1}{6} = 0.0043$
 - $p(q|D2) = p(\text{onion}|D2)p(\text{soup}|D2)p(\text{onion}|D2) = 0.3 * 0.156 * 0.3 = 0.014$
 - $p(q|D3) = p(\text{onion}|D3)p(\text{soup}|D3)p(\text{onion}|D3) = 0.033 * 0.022 * 0.033 = 0.00002$
 - Ranking $D2 > D1 > D3$
- Can we find a Dirichlet prior which will give the same scores?
 - Yes, only because the length of all documents is the same

LM VS. Prob. Model

- One difference is whether “Relevance” figures explicitly in the model or not
 - LM approach attempts to do away with modeling relevance (whereas this is the central variable in the BIM approach)
- LM approach assumes that documents and expressions of information needs are of the same type
- Computationally tractable, intuitively appealing

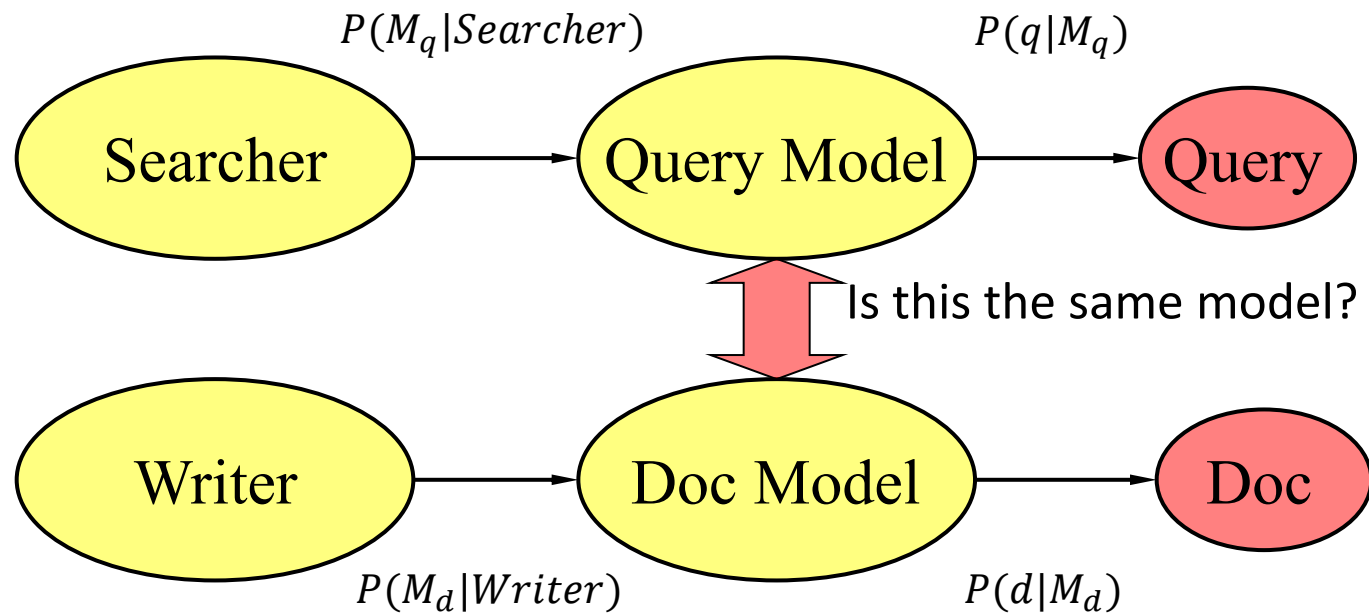
LM VS. Prob. Model

- Problems of basic LM approach
 - Assumption of equivalence between document and information needs representation is unrealistic
 - Very simple models of language
 - Relevance feedback is difficult to integrate, as are user preferences, and other general issues of relevance
 - Can't easily accommodate phrases, passages, Boolean operators

Pyserini Usage

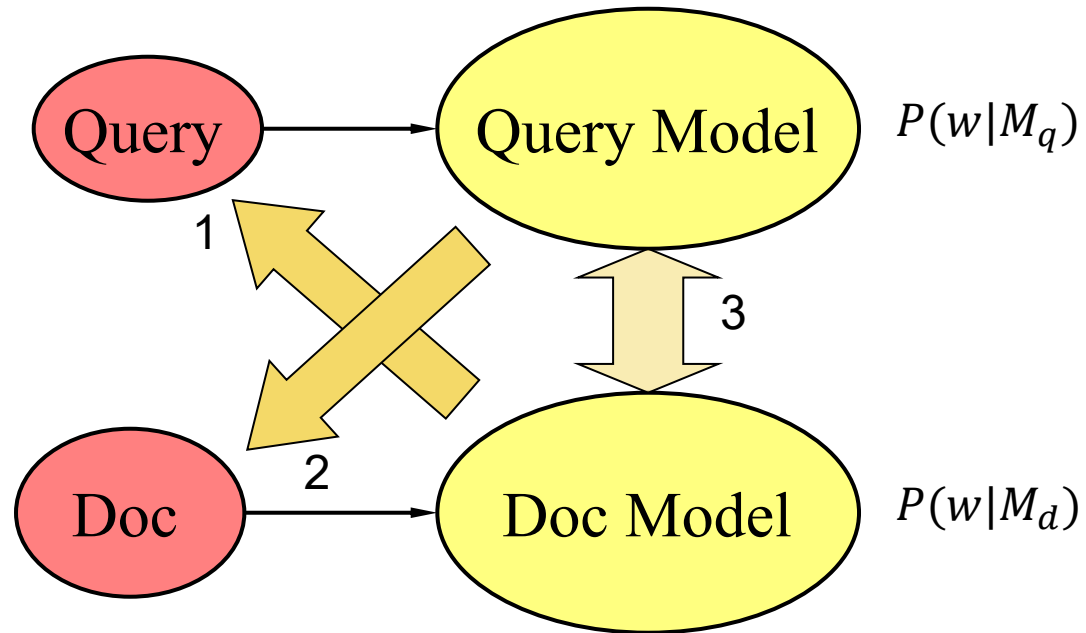
```
searcher = LuceneSearcher(index_path)
# specify custom analyzer for the query processing step to match the
way the index was built
analyzer = get_lucene_analyzer(stemmer='krovetz', stopwords=False) #
Ensure no stopwords are removed from the query
searcher.set_analyzer(analyzer)
# Optionally, configure BM25 parameters (can be adjusted as needed)
searcher.set_qld(mu=1000)
```

Alternative Models of Text Generation



Risk minimization framework [Lafferty&Zhai '01]

Retrieval Using Language Models



Retrieval: Query likelihood (1), Document likelihood (2), Model comparison (3)

Query Likelihood

$$p(q|M_d)$$

- Major issue is estimating document model
 - i.e., smoothing techniques instead of tf.idf weights
- Good retrieval results
- Problems dealing with relevance feedback, query expansion

Document Likelihood

$$p(d|M_q)$$

- Problem: query is short
- Potential solution: estimate a relevance model (query-expanded model)
 - More on this later...

Model Comparison

- Estimate query and document models and compare
- Suitable measure is **KL-divergence**

$$D(M_q || M_d) = \sum_w p(w|M_q) \log \frac{p(w|M_q)}{p(w|M_d)}$$

- Summation is over all words w in the vocabulary
- Equivalent to query-likelihood approach if simple empirical distribution (i.e., MLE) used for query model

Example – Ranking Using KL-Divergence

- Rank the following documents with respect to the query = “onion soup onion”
- Use JM smoothing with $\lambda = 0.2$

D1: onion vegetable soup vegetable mushroom corn

D2: corn onion soup onion mushroom corn

D3: potato pumpkin tofu potato tofu potato

Example

- First we estimate the collection LM - M_C
 - Total terms 18, unique terms 8

term	onion	vegetable	soup	mushroom	corn	potato	pumpkin	tofu
tf	3	2	2	2	3	3	1	2
P_{MLE}	3/18	2/18	2/18	2/18	3/18	3/18	1/18	2/18

Example

▪ Unsmoothed document LMs

term/doc.	onion	vegetable	soup	mushroom	corn	potato	pumpkin	tofu
D1	1/6	2/6	1/6	1/6	1/6	0	0	0
D2	2/6	0	1/6	1/6	2/6	0	0	0
D3	0	0	0	0	0	3/6	1/6	2/6

▪ Smoothed LM

- ($\lambda = 0.2$):
 - $p(\text{onion} | D1) = 0.8 * (1/6) + 0.2 * (3/18) = 1/6$
 - $P(\text{soup} | D1) = 0.8 * (1/6) + 0.2 * (2/18) = 0.156$

term/doc.	onion	vegetable	soup	mushroom	corn	potato	pumpkin	tofu
D1	1/6	0.29	0.156	0.156	1/6	0.033	0.011	0.022
D2	0.3	0.022	0.156	0.156	0.3	0.033	0.011	0.022
D3	0.033	0.022	0.022	0.022	0.033	0.433	0.144	0.29

Example – Query Likelihood

- Query likelihood
 - $p(q|D1)=p(\text{onion}|D1)p(\text{soup}|D1)p(\text{onion}|D1)=1/6*0.156*1/6=0.0043$
 - $p(q|D2)=p(\text{onion}|D2)p(\text{soup}|D2)p(\text{onion}|D2)=0.3*0.156*0.3=0.014$
 - $p(q|D3)=p(\text{onion}|D3)p(\text{soup}|D3)p(\text{onion}|D3)=0.033*0.022*0.033=0.00002$
 - Ranking: D2 -> D1 -> D3
- Can we find a Dirichlet prior which will give the same scores?
yes, only because all documents have the same length

Example – Ranking Using KL-Divergence

- Estimate M_q : $p(\text{onion} | M_q) = \frac{2}{3}$; $p(\text{soup} | M_q) = \frac{1}{3}$

$$KL(M_q | M_{D1}) = \frac{2}{3} \ln \frac{2/3}{1/6} + \frac{1}{3} \ln \frac{1/3}{0.156} = 1.177$$

$$KL(M_q | M_{D2}) = \frac{2}{3} \ln \frac{\frac{2}{3}}{0.3} + \frac{1}{3} \ln \frac{\frac{1}{3}}{0.156} = 0.785$$

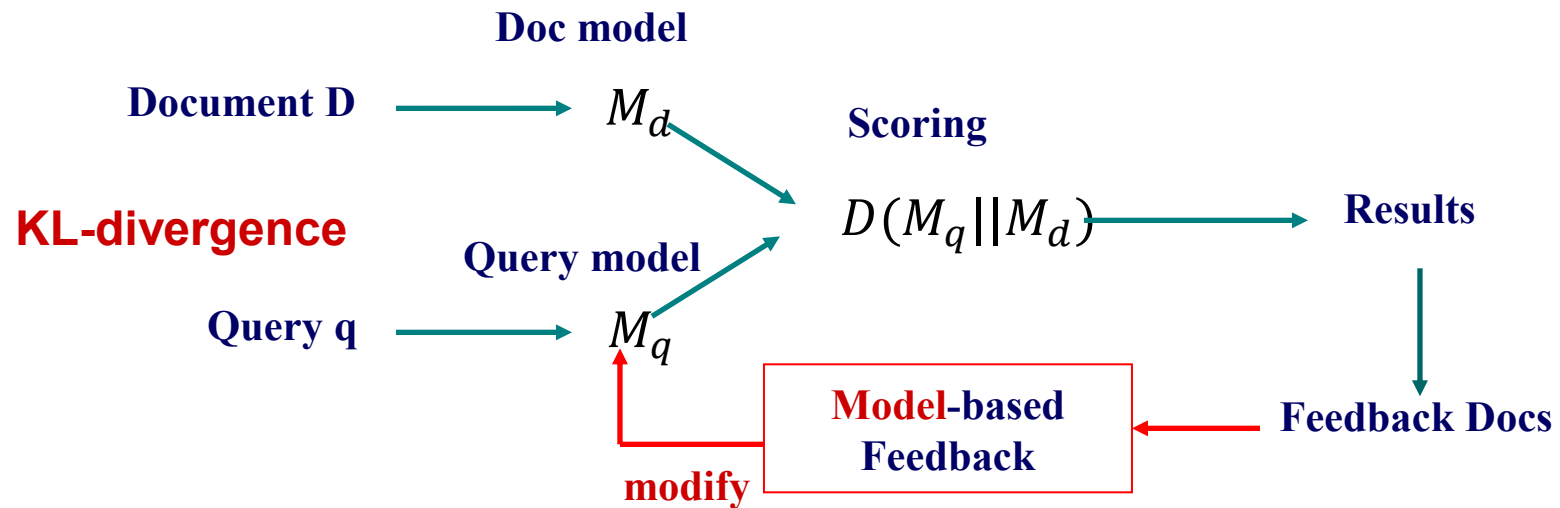
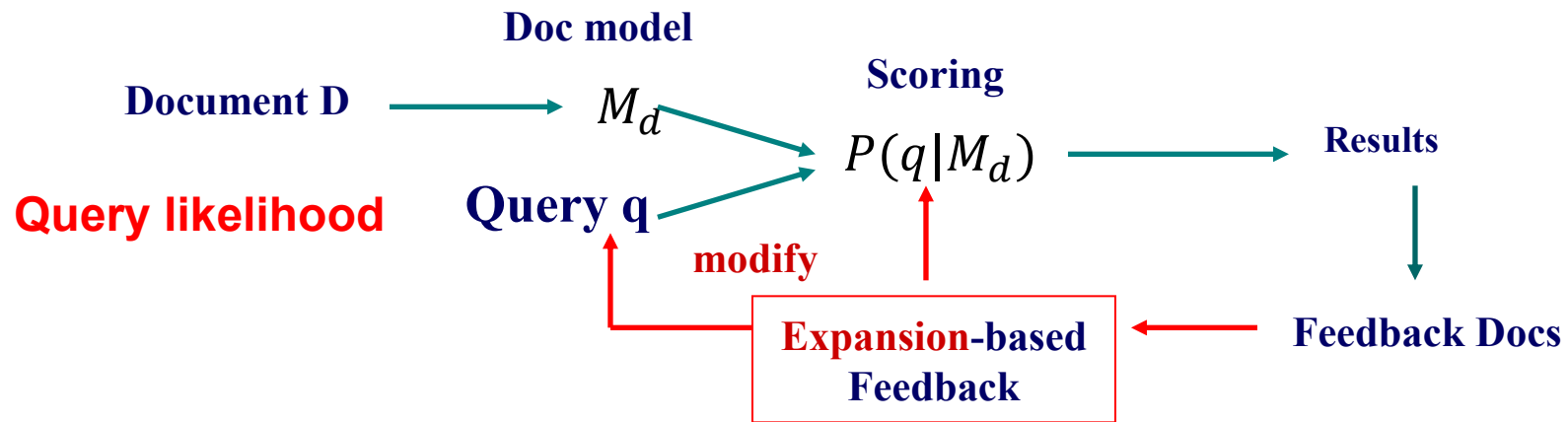
$$KL(M_q | M_{D3}) = \frac{2}{3} \ln \frac{\frac{2}{3}}{0.033} + \frac{1}{3} \ln \frac{\frac{1}{3}}{0.022} = 2.910$$

- Ranking by $-KL$: $D2 \rightarrow D1 \rightarrow D3$
- The same ranking as induced by the query-likelihood approach

Relevance Feedback Using Language Modeling Approach

- Introduce a query model & treat feedback as query model updating
 - Retrieval function:
 - Query-likelihood $\Rightarrow$ KL-Divergence
 - Feedback:
 - Expansion-based $\Rightarrow$ Model-based

Expansion-Based Vs. Model-Based



Comparison to Vector-Space Representation (tf.idf)

- LM probabilities are estimated using relative term frequencies
 - Reminiscent of tf
- Probabilities are based on length normalization of term frequencies
- Where is IDF?
 - In the smoothing

Comparison with Vector Space

- Differences
 - Based on probabilities rather than similarities
 - Intuitions are probabilistic rather than geometric
 - Details of use document length and term, document, and collection frequency differ

The Connection Between Smoothing and IDF

- For notational convenience, we will write $p(w|d)$ for $p(w|M_d)$
 - w : term, d : document, q : query with query-terms $q = q_1 \dots q_n$
 - $p_s(w|d)$ probability of word w that is seen in d
 - $p_u(w|d)$ probability of word w that is unseen in d
 - $tf(w|d)$ the count of word w in document d
- For query likelihood approach: score document d by
 - $p(q|d) = \prod_i p(q_i|d)$
 - $\log p(q|d) = \sum_i \log p(q_i|d)$

The Connection Between Smoothing and IDF

- $\log p(q|d) = \sum_i \log p(q_i|d) = \sum_{i:tf(q_i \in d) > 0} \log p_s(q_i|d) + \sum_{i:tf(q_i \in d) = 0} \log p_u(q_i|d)$
 $= \sum_{i:tf(q_i \in d) > 0} \log p_s(q_i|d) - \sum_{i:tf(q_i \in d) > 0} \log p_u(q_i|d) + \sum_{i:tf(q_i \in d) > 0} \log p_u(q_i|d)$
 $+ \sum_{i:tf(q_i \in d) = 0} \log p_u(q_i|d) = \sum_{i:tf(q_i \in d) > 0} \log \frac{p_s(q_i|d)}{p_u(q_i|d)} + \sum_i \log p_u(q_i|d)$

The Probability of Unseen Words

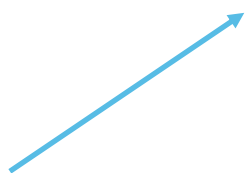
- Smoothing with the corpus $\mathcal{C}$
- $p_u(q_i|d) = \alpha_d p_{MLE}(q_i|\mathcal{C}) = \alpha_d \frac{tf(q_i \in \mathcal{C})}{\sum_{w'} tf(w' \in \mathcal{C})}$
 - α_d is a constant that depends on d

The Connection Between Smoothing and IDF

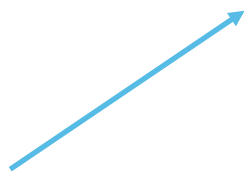
- $\log p(q|d) = \sum_i \log p(q_i|d) = \sum_{i:tf(q_i \in d) > 0} \log \frac{p_s(q_i|d)}{p_u(q_i|d)} + \sum_i \log p_u(q_i|d)$
 $= \sum_{i:tf(q_i \in d) > 0} \log \frac{p_s(q_i|d)}{\alpha_d p_{MLE}(q_i|C)} + n \log \alpha_d + \sum_i \log p_{MLE}(q_i|C)$

The Connection Between Smoothing and IDF

$$\begin{aligned} \blacksquare \log p(q|d) &= \sum_i \log p(q_i|d) = \sum_{i:tf(q_i \in d) > 0} \log \frac{p_s(q_i|d)}{p_u(q_i|d)} + \sum_i \log p_u(q_i|d) \\ &= \sum_{i:tf(q_i \in d) > 0} \log \frac{p_s(q_i|d)}{\alpha_d p_{MLE}(q_i|C)} + n \log \alpha_d + \sum_i \log p_{MLE}(q_i|C) \end{aligned}$$



plays the “IDF” role



document independent

Efficient implementation
with inverted index!